

EXERCÍCIO PRÁTICO

Galeria 3D: criação e instanciação de objetos

Computação Gráfica • Python + PyGame • Aula 8

Desafio

Construir uma pequena galeria interativa que exiba diferentes instâncias de uma mesma malha 3D. O estudante deverá observar como uma única geometria pode gerar objetos visualmente distintos por meio de escala, rotação, posição, materiais/cores e organização hierárquica.

Nome: _____ Data: ____/____/____

Curso/ turma: _____

1. Objetivos de aprendizagem

- Representar um objeto 3D por vértices, arestas e faces.
- Criar uma malha reutilizável e instanciá-la várias vezes na cena.
- Aplicar transformações geométricas — translação, rotação e escala — em objetos.
- Compreender a relação entre objeto, câmera virtual, projeção e tela 2D.
- Organizar uma cena com objetos independentes e um objeto composto pai-filho.
- Implementar interação por teclado e feedback visual de seleção.

2. Contextualização

Em uma aplicação gráfica, a geometria descreve a forma, enquanto o estado de cada instância determina onde ela está, qual é seu tamanho e como está orientada. Separar malha e instância evita duplicação e permite construir cenas maiores com poucos dados geométricos. Nesta atividade, a galeria será um laboratório para experimentar esse princípio.

Ideia central

Malha = “o que é o objeto”. Instância = “como e onde o objeto aparece”. Uma mesma malha pode representar uma escultura pequena, uma coluna alta ou parte de um objeto composto.

3. Enunciado

Desenvolva, em Python com PyGame, uma aplicação chamada Galeria 3D. A cena deverá apresentar pelo menos três objetos derivados da mesma malha de cubo, posicionados lado a lado como peças de uma exposição. Cada instância deverá possuir transformações próprias e, ao menos uma delas, deverá girar continuamente.

A aplicação também deverá permitir selecionar uma peça e modificar a visão da câmera. O programa deve exibir instruções de controle e destacar visualmente o objeto selecionado. Como extensão obrigatória da organização da cena, acrescente um objeto composto formado por um corpo e uma peça filha, como uma haste, placa ou pedestal.

4. Conceitos mobilizados

4.1 Malha poligonal

Uma malha é formada por uma lista de vértices e uma lista de faces. Cada vértice contém coordenadas (x, y, z). Cada face referencia índices de vértices e define um polígono. O cubo é adequado para a atividade porque sua estrutura é simples, mas o mesmo procedimento pode ser aplicado a prismas, pirâmides e modelos importados.

4.2 Instanciação

A função de desenho recebe a mesma malha e parâmetros diferentes de posição, escala e rotação. Assim, a geometria é definida uma vez e reutilizada. Essa separação favorece desempenho, manutenção e consistência visual.

4.3 Transformações

A translação desloca o objeto; a escala altera suas dimensões; a rotação muda sua orientação. A ordem das transformações pode modificar o resultado: girar antes de transladar não produz necessariamente a mesma cena que transladar antes de girar.

4.4 Hierarquia pai–filho

Em um objeto composto, a peça filha herda a transformação do pai e ainda pode possuir uma transformação local. Se o corpo de um robô se desloca, seu braço deve acompanhá-lo. Na galeria, use essa ideia em um pedestal com placa ou em uma escultura com um elemento articulado.

4.5 Câmera e projeção

Como a janela é 2D, os pontos 3D precisam ser projetados. Na perspectiva, objetos mais distantes aparecem menores. Uma forma simplificada é $x' = f \cdot x/z$ e $y' = f \cdot y/z$, em que f é a distância focal e z é a profundidade em relação à câmera.

5. Requisitos funcionais

- Janela de aproximadamente 1000 × 650 pixels e título identificando a Galeria 3D.
- No mínimo três instâncias da mesma malha, com posições e escalas distintas.
- Pelo menos uma rotação contínua e uma variação de cor por face ou por objeto.
- Seleção por teclas 1, 2 e 3, com contorno, marcador ou mudança de cor.
- Controle da câmera pelas setas ou por outro conjunto de teclas documentado.
- Alternância entre modo sólido e modo arame usando a tecla Espaço.
- Encerramento seguro pelo ESC e pelo botão de fechar a janela.
- Texto na tela com os controles disponíveis.

6. Código-base funcional

O código a seguir fornece uma base completa para a cena, incluindo a definição da malha, projeção perspectiva, instâncias, seleção e controle da câmera. O estudante deve executar, compreender e modificar o programa — não apenas copiá-lo.

```
import pygame, math
from pygame.locals import QUIT, KEYDOWN, K_ESCAPE, K_LEFT, K_RIGHT, K_UP, K_DOWN, K_w, K_s, K_SPACE, K_1, K_2, K_3

pygame.init()
TELA = pygame.display.set_mode((1000, 650))
pygame.display.set_caption("Galeria 3D — Aula 8")
RELOGIO = pygame.time.Clock()

# Oito vértices e seis faces de um cubo unitário
VERTICES = [(-1,-1,-1),(1,-1,-1),(1,1,-1),(-1,1,-1),
             (-1,-1,1),(1,-1,1),(1,1,1),(-1,1,1)]
FACES = [(0,1,2,3),(4,5,6,7),(0,4,7,3),
          (1,5,6,2),(3,2,6,7),(0,1,5,4)]
CORES = [(220,80,80),(80,160,230),(90,190,120),
          (235,180,70),(170,100,210),(80,200,200)]

def rotacionar(v, ax, ay):
    x,y,z = v
    c,s = math.cos(ax), math.sin(ax)
    y,z = y*c-z*s, y*s+z*c
    c,s = math.cos(ay), math.sin(ay)
    x,z = x*c+z*s, -x*s+z*c
    return (x,y,z)

def projetar(v, camera_yaw, camera_pitch, escala=150):
    x,y,z = rotacionar(v, -camera_pitch, -camera_yaw)
    z += 8 # distância da câmera
    if z <= 0.1: return None
    return (500 + escala*x/z, 330 - escala*y/z, z)

def desenhar_objeto(pos, escala, rot, selecionado=False):
    pontos=[]
    for v in VERTICES:
        local=rotacionar((v[0]*escala[0],v[1]*escala[1],v[2]*escala[2]), rot[0], rot[1])
        pontos.append(projetar((local[0]+pos[0],local[1]+pos[1],local[2]+pos[2]), yaw, pitch))
    if any(q is None for q in pontos): return
    ordem=sorted(range(6), key=lambda f: sum(pontos[i][2] for i in FACES[f])/4, reverse=True)
    for f in ordem:
        poligono=[pontos[i][2] for i in FACES[f]]
        pygame.draw.polygon(TELA, CORES[f], poligono)
        pygame.draw.line(TELA,(35,35,45),poligono[-1],poligono[0],2)
        for a,b in zip(poligono,poligono[1:]): pygame.draw.line(TELA,(35,35,45),a,b,2)
    if selecionado:
        pygame.draw.circle(TELA,(255,255,255), (int(pontos[6][0]),int(pontos[6][1])), 8, 2)

objetos=[{'pos':(-3,0,0),'escala':(1,1,1),'rot':(0,0)},
          {'pos':(0,0,0),'escala':(1.4,1.4,1.4),'rot':(0,0)},
```

```

    {'pos':(3,0,0),'escala':(1,2,1),'rot':(0,0)}]
yaw=pitch=0.0; angulo=0.0; selecionado=1; arame=False; executar=True
while executar:
    for evento in pygame.event.get():
        if evento.type == QUIT: executar=False
        if evento.type == KEYDOWN:
            if evento.key == K_ESCAPE: executar=False
            if evento.key == K_LEFT: yaw -= .1
            if evento.key == K_RIGHT: yaw += .1
            if evento.key == K_UP: pitch += .1
            if evento.key == K_DOWN: pitch -= .1
            if evento.key == K_SPACE: arame = not arame
            if evento.key in (K_1,K_2,K_3): selecionado = evento.key-K_1
    angulo += .01
    TELA.fill((18,25,40))
    pygame.draw.rect(TELA,(45,55,72),(80,500,840,80))
    for i,obj in enumerate(objetos):
        r=(0,angulo if i==1 else 0)
        desenhar_objeto(obj['pos'],obj['escala'],r,i==selecionado)
    fonte=pygame.font.SysFont('arial',20)
    texto='Galeria 3D | 1-3 selecionar | setas câmera | ESPAÇO: modo arame | ESC: sair'
    TELA.blit(fonte.render(texto,True,(235,235,235))),(90,610))
    pygame.display.flip(); RELOGIO.tick(60)
pygame.quit()

```

7. Tarefas de desenvolvimento

1. Execute o código-base e identifique no programa as listas de vértices, faces, cores e objetos.
2. Altere as posições e escalas das três instâncias. Descreva o efeito observado.
3. Implemente o modo arame de fato: quando arame for verdadeiro, desenhe apenas as arestas de cada face.
4. Adicione um objeto composto: desenhe uma segunda malha usando a posição e a rotação do objeto principal como referência.
5. Crie um painel simples com o nome da peça selecionada e suas transformações atuais.
6. Acrescente uma quarta instância com uma transformação não uniforme, por exemplo escala (0.7, 1.8, 0.7).
7. Teste diferentes ordens de rotação e explique como elas alteram a aparência da cena.
8. Registre duas capturas de tela: uma em modo sólido e outra em modo arame.

8. Questões para reflexão

9. Por que reutilizar uma malha é mais eficiente do que criar uma lista de vértices para cada objeto?
10. Qual é a função da profundidade z na projeção perspectiva? O que ocorre quando z se aproxima de zero?
11. Em que situação uma transformação não uniforme pode deformar um objeto de maneira indesejada?
12. Como você faria para que a peça filha girasse em torno do próprio eixo sem abandonar o pai?
13. Quais elementos do programa pertencem ao modelo da cena e quais pertencem à interface/interação?
14. Como a seleção poderia ser implementada com o mouse em vez de teclas?

9. Extensões opcionais

- Adicionar piso, paredes e iluminação simulada por gradientes ou cores dependentes da orientação das faces.
- Criar um catálogo com nomes, descrições e valores de transformação para cada peça.
- Implementar zoom e aproximação da câmera.
- Substituir o cubo por uma pirâmide ou por uma malha carregada de arquivo.
- Adicionar animação hierárquica: uma placa que oscila, gira ou acompanha o pedestal.
- Criar uma função genérica para desenhar qualquer malha convexa, não apenas o cubo.

10. Entrega esperada

Entregue o programa Python (.py) e um relatório curto (1 a 2 páginas) contendo: descrição da cena; esquema ou tabela das instâncias; explicação das transformações; resposta às questões 1 a 6; duas capturas de tela; e uma reflexão final sobre a reutilização de malhas.

11. Critérios de avaliação

Critério	Peso
----------	------

Modelagem e representação da malha	20%
Instanciação e uso correto de transformações	20%
Câmera, projeção e organização da cena	20%
Interação, seleção e feedback visual	15%
Objeto composto e hierarquia pai-filho	15%
Clareza do código, relatório e reflexão	10%

12. Fechamento

Conexão com a disciplina

A atividade integra modelagem, representação, transformações, câmera, projeção, hierarquia e interação. O foco não é produzir uma engine completa, mas tornar visível o pipeline: dados geométricos → transformação do objeto → transformação da câmera → projeção → rasterização na janela.

Professor(a): _____ Nota: _____